Speech recognition — making it work for real

F Scahill, J E Talintyre, S H Johnson, A E Bass, J A Lear, D J Franklin and P R Lee

Speech recognition algorithms have now progressed to the point where large-vocabulary and continuous-speech recognition are frequently demonstrated in the laboratory. This paper describes the implementation of the BT speech recogniser known as Stap, a suite of software in which the latest speech recognition algorithms are implemented in a flexible, robust package that is portable to a wide range of platforms.

1. From algorithms to services

Previous speech recognisers developed by BT concentrated on accurate recognition of small vocabularies across the telephone network, e.g. recognition of 'yes' and 'no' or the digits '0' to '9' using a whole word model for each word. These recognisers suffer from the disadvantage that changes to the vocabularies require large collections of speech data to train models for the new words. Even with this drawback, these recognisers can still support many useful applications, such as BT CallMinder ™.

Speech recognition algorithms have now progressed to the point where large-vocabulary and continuous-speech recognition are frequently demonstrated in the laboratory. The goal now is to make these algorithms available for use in real interactive speech services.

1.1 Flexible vocabularies

Developments in sub-word unit algorithms for speech processing mean that it is now possible to quickly create vocabularies for new applications or change vocabularies for existing ones, without additional data collections and model training. Some applications in fact depend on the ability to change the recogniser vocabularies on a day-to-day, or even minute-to-minute basis!

1.2 Large vocabularies

The accuracy and flexibility of recognition algorithms has improved such that it is now possible to offer services with active[1] vocabulary sizes of up to 1000 words. Such systems would have been impractical with previous speech recognition technology.

1.3 New services

The increases in vocabulary flexibility and vocabulary size have led to a huge range of trial automated services such as call centre automation, e.g. bill payment by phone, automated database enquiries and advanced voice mail services.

These services require a speech recogniser in which the latest speech recognition algorithms are implemented in a flexible, robust package of software that is portable to a wide range of platforms, including the BT/Ericsson interactive speech applications platform (ISAP) [1] and Unix® workstations.

This paper describes such an implementation, known as Stap, and outlines how it was designed and how its capabilities meet the requirements of current and future interactive speech services.

2. Speech recognition — a brief overview

2.1 Vocabulary

A speech recogniser is designed to recognise one of a set of words or phrases specified in a vocabulary. The recogniser is presented with a segment of sound, usually several seconds long, which henceforth will be called an utterance. The recogniser attempts to distinguish the speech from the noise, and to label the utterance as matching one of the items in its vocabulary.

[1] The active vocabulary is that which can be recognised at any one time. For example a recogniser could be capable of recognising any of 10 000 town names, but be set to only expect one of 1000 in answer to a specific query.

2.2 HMMs

Many speech recognisers, including Stap, use hidden Markov models (HMMs) to represent their vocabularies. An HMM is a statistical model [2]. Each HMM represents a unit of sound, such as a word or a sub-word unit (e.g. phoneme) that is to be recognised. Each model is constructed using statistics calculated from examples of the speech unit that the model represents. This process is referred to as ‘training’ the models. The speech used for training should be representative of the speech that the recogniser will encounter when it is used. In some cases this is not possible and it becomes necessary to modify the models to suit the data, this is known as ‘adaptation’. Some speech recognisers require that the speech data be provided by a single person, these are ‘ speaker-dependent’ and will only recognise that person’s speech. ‘ Speaker-independent’ systems are those that combine the training data from enough speakers to work well for almost any person’s speech.

2.3 Front end

To improve accuracy and efficiency, HMMs do not model the speech directly. Instead, they model a transformed version of the speech which is more compact and decorrelated. This transformation process is known as ‘feature extraction’ and is performed by the ‘ front end’ (FE).

2.4 Parser

To perform the actual recognition match, a ‘ recognition network’ of models (sub-word or whole word) is constructed to represent the entire vocabulary. The recogniser’s task is to find the path, or paths, through the network which most closely match the utterance. The recognition component which searches for these paths is the ‘ parser’.

2.5 Rejection

Speech recognition is difficult — no speech recogniser, not even a person, is 100% accurate. There are many reasons why an utterance may be mis-recognised. For example, the person speaking may have an accent on which the recogniser has not been trained, or there may be a high level of background noise from a busy office. Alternatively the person speaking may simply have said something that was not in the recogniser’s vocabulary. An important attribute of any speech recogniser is therefore a ‘ rejection’ capability. Rejection is the means by which a recogniser checks its confidence in the recognition match, so that the speaker can be asked to confirm the result if necessary .

2.6 Pause detection

Vocabularies can contain words or phrases of varying length. The user will speak the word or phrase an unknown amount of time after being prompted to speak. The speech recogniser cannot therefore rely on listening for a fixed period of time. It must accurately determine when the person has finished speaking so that the matching process can be stopped and the recognition result returned. Inter-word and intra-word pauses must be ignored. Henceforth this activity will be referred to as ‘ pause detection’ .

3. What is Stap?

Stap is a package of software for performing advanced speech recognition. It comprises two main tools:

- StapRec — a real-time advanced speech recogniser,
- StapVoc — a recognition network creation tool.

A number of additional tools for more advanced users are also provided; these include:

- grok — a network creation tool which accepts any BNF style grammar,
- Xpressive — an X-window interface to Stap

3.1 StapRec

StapRec is:

- speaker-independent, i.e. it works for any speaker without prior training or enrolment,
- able to accept input from the telephone network or a workstation’s audio system,
- ‘real time’ (speech data is processed as it is received) — the recogniser will normally give out an answer as soon as it has detected that the speaker has finished speaking,
- multi-language — currently British English and American English are supported, other languages being added by creating new StapRec data files,
- flexible — the vocabulary is defined by a recognition network which may consist of isolated words, short phrases, connected digits, alphanumerics or any combination thereof, with the active vocabulary size varying from 1—1000 words , which can be subsetted or completely replaced very quickly.

3.2 StapVoc

StapVoc is:

- a vocabulary generator that converts textual vocabulary specifications into recognition networks and the associated files necessary to configure and control a StapRec,
- easy to use by application developers without the need for prior speech recognition knowledge.

3.3 Key Issues

In addition to the functional requirements for speech recognition, Stap was designed with a number of other key issues in mind:

- Portability and maintainability

  Computing hardware is changing at an increasing rate. New and more powerful processors are constantly becoming available. Similarly the requirements on and capabilities of speech recognition algorithms are constantly improving. It is therefore essential that Stap is portable and maintainable.

  To this end Stap was specified and designed using the Booch object oriented design methodology [3], and implemented in C++ [4]. The Booch notation for expressing object-oriented designs is briefly described in the Appendix.

- Cost

  The commercial viability of an automated interactive speech service is largely dependent on the cost per channel. Speech recognition is one of the most expensive facilities that a speech service may have to provide. Therefore a primary goal for any speech recogniser is memory and computational efficiency. This allows useful speech recognition to be provided on low power hardware, yet permits more powerful hardware to make a trade-off between handling more channels simultaneously and performing more accurate speech recognition.

- Ease of use

  The rapid and widespread deployment of speech recognition can only be achieved if there is a simple application developer interface. Application developers should not require an understanding of hidden Markov models to add interactive speech to their services.

4. StapRec

4.1 The recogniser architecture

Figure 1 shows a conceptual overview of the main components of StapRec.

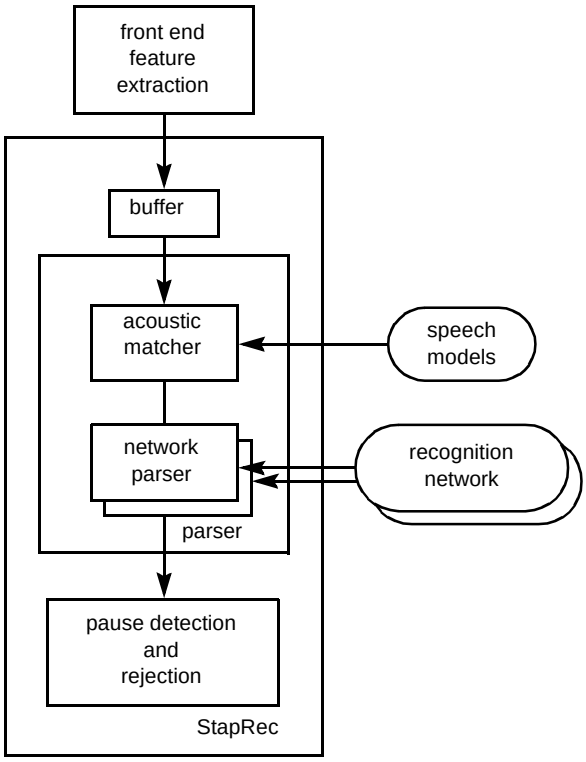


Fig 1 Components of a speech recogniser.

Feature extraction

Speech arriving at the front end is already in digital format, for example 64 kbit/s A-law for ISAP based applications. The feature extraction block performs the signal processing required to reduce the speech signal to a more compact and easier to recognise parameterisation. StapRec uses a combination of Mel frequency cepstral co-efficients (MFCC), delta MFCCs and delta log energy [5, 6].

As can be seen from Fig 1, the feature extraction block is separate from StapRec. This allows for easy replacement with alternative parameterisations for efficiency or accuracy improvements. It also allows for the different speech formats that can be encountered when Stap is run in a workstation environment. Furthermore the front end can be run on a separate processor, such as a digital signal processor (DSP), with better price/performance ratio for the type of vector maths processing required.

Variable frame rate

The parameterisation process initially generates a frame of data every 16 ms. The number of frames is then adjusted to give variable frame rate (VFR) data.

So that timing information is not lost, the VFR process attaches a number to each retained frame, indicating how many real frames it represents. Using VFR data reduces the amount of processing required during recognition, and also

emphasises the more discriminatory changing regions of the signal.

Buffering

The rate at which StapRec can process the frames from the FE depends on a number of factors including the CPU hardware and recognition vocabulary size. The processing load also varies during the course of recognising an utterance, depending on the proportion of the recognition network that is active. To allow for these variations StapRec maintains a separate buffer process between the front end and the parser. In principle, this buffering could be integral to the FE, but, in practice, this causes problems with DSP implementations of the front end due to memory limitations.

The separation of the buffer into a separate Unix process also allows StapRec to receive data from the FE before it has completed configuring. This can be important when switching between large recognition vocabularies on the fly.

Parser

StapRec extracts data from the buffer process and passes it, one frame at a time, to the recognition parser which performs the detailed pattern match. The parser matches the VFR feature data against the HMM speech models, and combines the outputs of this match with the recognition network which defines the vocabulary to be recognised. The parser will be described in more detail later.

The separation of the acoustic match and network parse enables StapRec to efficiently [1] support multiple parallel network parsers recognising the same feature data. This leads to a number of advantages.

- Faster reconfiguration where vocabularies overlap — here, an applications vocabulary can be split into shared and independent regions. When the recogniser is switched between vocabularies then only the independent regions need to be reloaded. Thus, re-configuration time is reduced.

- Multiple independent matches available — normally a real recogniser must be able to detect silence and out-of-vocabulary (OOV) utterances in addition to finding the best matching vocabulary item. This is usually done through the use of silence and OOV recognition paths. If these were incorporated in the same network as the vocabulary then the network parser design would be complicated since it would have to treat these specially. The parallel network design enables a simpler parser design while providing a more generic functionality.

- Possible parallel processing — for very complex recognition tasks a single CPU may not be powerful enough for real time recognition. Parallel networks enable us in the future to easily distribute large recognition tasks across multiple CPUs.

Pause detection and rejection

After each frame has been processed by the parser, StapRec checks to see whether the speech has finished and, if it has, extracts the recognition results from the parser. A number of confidence measures are then applied to the results and, dependent on these, a decision is made — typically ‘accept’, ‘reject’ or ‘query’.

To ensure good performance the pause detection and rejection software make use of both raw signal measures (e.g. SNR) together with the quality of match measures derived from the recognition results.

This mix of features means that the recogniser is capable of detecting errors caused by:

- out-of-vocabulary speech,
- noisy lines,
- poor pronunciation,
- confusable vocabulary words.

4.2 Advanced facilities

StapRec includes some advanced facilities which are not normally found in speech recognition software. These facilities have been developed to meet the requirements of BT’s own advanced speech applications.

Vocabulary weighting and subsetting

For many large-vocabulary tasks such as automated Directory Enquiries, there is a considerable amount of information regarding the *a priori* probability of a particular word in the vocabulary being used [7]. Overall accuracy can be substantially improved if these probabilities are used to weight paths in the recognition grammar. StapRec therefore provides a facility where a probabilistic weighting can be applied to items in the vocabulary. In its simplest form this provides a mechanism for subsetting the vocabulary by turning off some vocabulary items altogether, i.e. weighting them to zero. The advantages of this facility are clear — any reduction in the vocabulary size will increase accuracy and reduce the computational cost.

[1] Since the CPU-expensive parts of the HMM calculations are performed only once.

The application developer controls the vocabulary weighting by passing a simple text list of the vocabulary words and their weightings. The weightings are applied by modification of node penalties internal to the recognition network. It is performed at the start of each recognition and the network is reset at the end of the recognition. There is no significant increase in recognition time for performing this process.

Unlike other commercial speech recognisers which provide similar facilities, Stap provides the control down to specific words and can perform soft weighting (use of *a prioris*) or hard subsetting (setting some weightings to zero and others to one).

Homophones

StapRec, like all speech recognisers, is only able to recognise what a particular word sounds like. This is a problem when two or more words in a vocabulary sound the same. These are known as homophones, e.g. brake and break[1].

It is inefficient for the recognition network to contain separate copies of the homophones since this leads to unnecessary duplication of calculations. Instead, a single path in the grammar is used to represent the homophones and this path is expanded to include all the associated vocabulary items.

The mapping of network paths to the vocabulary items is produced when the recognition network is generated by StapVoc. Since StapVoc knows the phonetic transcription of the words, it is able to detect homophones and relabel those paths in the network with a ‘homophone tag’. A separate mapping between these tags and the real vocabulary items is updated and stored.

The advantage of this approach is that the application developer needs to take no special measures to cope with homophones. Stap compresses and expands them automatically and invisibly.

4.3 Class diagram

Figure 2 shows how the basic components of StapRec translate into a class design using the Booch notation [2].

At the centre of the architecture is the RecogController. This co-ordinates and delegates the other components to recognise an utterance encoded as a sequence of frames.

The Recogniser class manages the multiple network parsers, ensuring that their recognition matches are combined in sensible ways and that all the parsers are synchronised. The ModelSet is the repository for all data relating to the HMMs that is not specific to their context in the network. To allow for integration of other types of network parser with Stap the Network class is decoupled from the rest of StapRec through an interface class VegaIF.

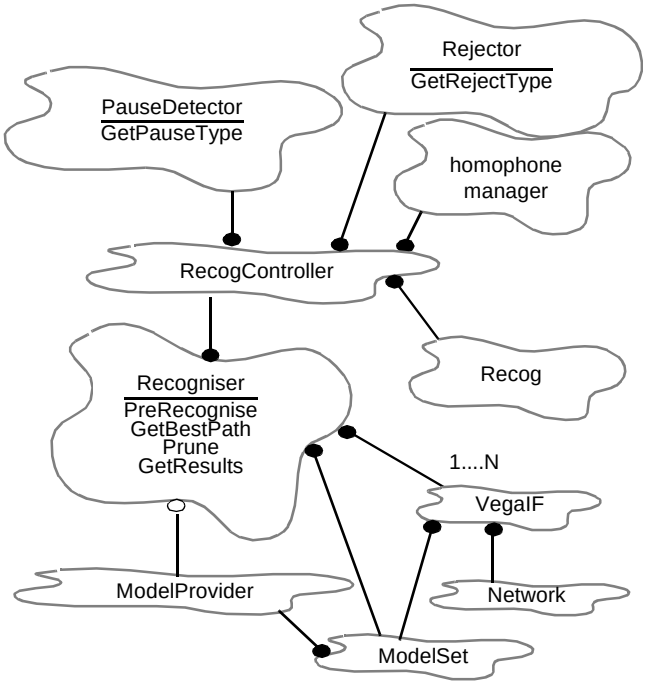


Fig 2 Main architectural components of StapRec.

StapRec detects when the caller has finished speaking as an integral part of the recognition process. Hence, after each frame is processed by the Network objects, the PauseDetector must examine the current recognition result, together with some raw signal data, to decide whether the caller has finished speaking. When the end of speech has been detected, the results from the Networks are combined by the Recogniser and passed to the Rejector for checking. Finally the RecogResults are returned to the applications via the RecogController.

5. Vega — the parser

The heart of StapRec is the acoustic matcher and grammar parser. The collection of classes that perform these roles is known as Vega. As the principle of the algorithm-matching process is adequately covered by other papers [5, 6], this section concentrates on the functionality and design of the parser.

[1] This problem is most commonly found when automating access to large databases, where the application developer has little control over the contents of the database.

[2] See the Appendix for a brief explanation of Booch.

5.1 Networks, nodes and tokens

The parser design is based upon three main concepts — networks, nodes and tokens.

Networks

In Vega, the speech recognition vocabulary is represented as a finite network (or graph) of nodes defining the HMM interconnections. This representation allows considerable generality and flexibility.

For example, a network constructed by StapVoc, for the recognition of a constrained set of words or phrases, has the property that (roughly speaking) no node has two distinct successor nodes having the same model. This leads to a tree-like topology where common prefixes are shared between paths in the network. The advantages of which are:

- fewer nodes in the network — improved memory efficiency,
- shared calculations — improved CPU efficiency.

Figure 3 shows a simple example for the vocabulary duck, dog, calf and cat represented by sub-word unit models.

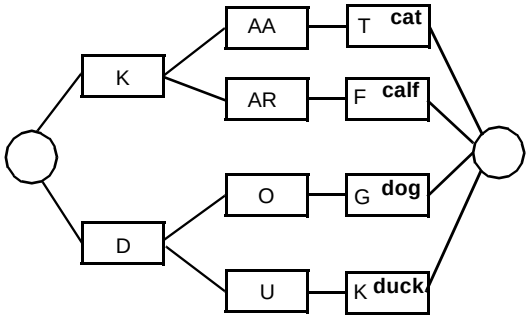


Fig 3 Simple StapVoc generated network topology.

A useful feature of this network is that each vocabulary item corresponds to a single terminal (leaf) node. This has two advantages:

- the match scores for all of the vocabulary can be made available to the application (since all vocabulary items are represented by a unique path through the network),
- vocabulary items can be turned off (subsetting) or probabilistically weighted by the application of simple penalties to the terminal nodes.

Other recognition tasks require different network topologies. For unconstrained connected digit recognition a tree structure is neither practical nor desirable. Here a more sensible network is one where paths can merge and can loop back to a previous node (see Fig 4).

Vega accepts arbitrary network topologies and importantly, arbitrary network sizes. In principle, the size of network that Vega can load is limited only by the memory available on the platform. This is typically up to 64 Mb which is sufficient for recognition vocabularies of 100 000 words containing more than a million nodes.

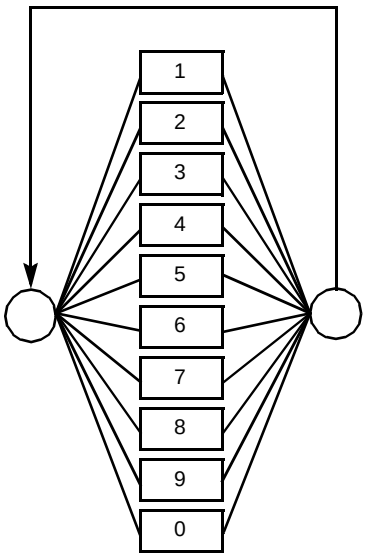


Fig 4 Unconstrained connected digit network topology.

Nodes

A node is the basic processing unit in speech recognition. It normally has a model attached to it for recognising a single phoneme or word, but alternatively can be just a connection point (null node — see below).

A null node is useful when handling many-to-many network connections. A null-node replaces these types of connections by a many-to-one and one-to-many connections. Thereby reducing token propagation from N^2 to 2N without loss in generality. The structure of a node is shown in Fig 5.

As can be seen, the node is split into static and dynamic data. For memory-efficient recognition with network sizes of up to a million nodes, each node must occupy as little memory as possible. In Vega the static node data is represented typically [1] in 50 bytes of memory, while the dynamic data is allocated from a central pool only if the node becomes activated, and is returned to the pool if the node is deactivated.

[1] Actual space used is dependent on the number of output edges from the node.

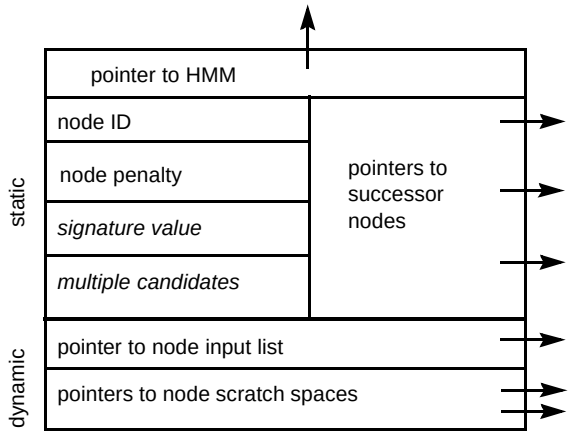


Fig 5 Structure of a node.

The node design supports multiple node scratch spaces which, when combined with the node signature, provide an efficient mechanism for multiple candidate processing [8].

Tokens

The network, and the nodes from which it is made up, describe the vocabulary of the speech recogniser. During recognition the input speech is matched against the network of nodes. Each frame that is processed causes a token to be emitted from each node. Before the subsequent frame is processed tokens move from a node to successor nodes in the network. For example, in Fig 3 a token would be emitted from the 'D' node into the 'O' and 'U' nodes.

Figure 6 shows the structure of a token. The pointer to the node gives the identity of the last model matched and the time index indicates when this occurred. By tracing back through the pointers to previous tokens a sequence of matched models is given.

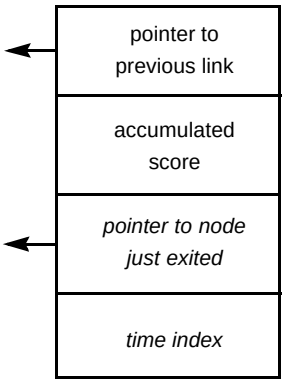


Fig 6 Structure of a token.

5.2 *Overview of the pattern matching process*

For HMM-based speech recognition the matching process can be split into two areas:

- acoustic match,
- network parse.

Acoustic match

The acoustic match is given by calculating the HMM state output probabilities. This involves the evaluation of multi-modal, multi-variate probability density functions and so is particularly CPU expensive. Fortunately, the probabilities are dependent only on the current frame of data and the HMM parameters and so need only be calculated once per frame for each HMM. The state probabilities are then available to the HMM instances associated with each node in the network.

Network parse

The network parse can be viewed at two levels:

- inter-node parse,
- intra-node parse.

For the inter-node parse Vega embodies single-pass Viterbi recognition using the token-passing mechanism [9]. Information is passed between nodes using the tokens described in the previous section. A token contains information on the progress of the recognition up to a particular point in time. With each incoming frame of speech, tokens waiting at node inputs are passed into the node's model instance. Each model instance updates its state scores[1] according to the new frame and the node may then emit a new token. The emitted tokens are propagated along network edges to wait again at node inputs for the following frame.

The score in the token can be modified when it traverses the edges, as required by the vocabulary weighting and sub-setting described earlier. The score can also be modified on entry into a node, as required, to implement inter-word penalties such as bi-gram language models.

[1] Each state score in the model instance may have been derived from a separate input token. For example in an N-state model, the score in state N may be based on the token input at $t-N$ whereas the score in state 1 may be based on the token input at time $t-1$.

Figure 7 shows the state of a recognition network at the point that a new output token is created by the first node in the network and propagated to the successor nodes. It shows how the output token points to both the input token from which it is derived, and the node that created it.

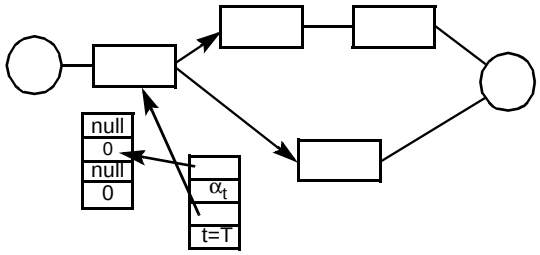


Fig 7 Network state when first token is emitted from the first node with score α_t after T frames of data.

Tokens emitted from the final node in the network represent the best match between the data processed so far and the recognition vocabulary. The recognition result is obtained by tracing back through the linked list of tokens.

Intra-node processing requires the updating of the state scores in the node's model instance. The state scores are maintained in an array (node scratch space) that is dynamically allocated when the node is activated. Each state score is obtained by combining the acoustic match score for the current frame (state output probability) with the scores from preceding states in model instance together with the state transition probability matrix.

The scratch space has the form shown in Fig 8. Since simple left-to-right (Bakis) HMM models are used within Vega, the scratch space is efficiently updated by processing the states in reverse order N down to 1.

State Index	1	2	3	..	N
Token Pointer					
State Score					

Fig 8 Scratch space form.

Unlike the inter-node processing, neither the time index nor the identity of the best preceding state are recorded when state scores are updated.

5.3 Efficiency considerations

Configuration time is an important issue for speech recognisers running on platforms supporting multiple applications and multiple channels (e.g. ISAP). To ensure that the recognisers can be efficiently utilised it must be possible to quickly change recognition vocabularies.

To improve configuration time, Vega supports:

- fast transfer between memory and file of the recognition network and HMMs in an internal binary format,
- model loading on demand — Vega loads new models only as required, with, in general, vocabulary changes involving new networks that contain arrangements of HMMs different from those that have already been loaded,
- recognition network pre-compilation — tools such as StapVoc and grok ensure that the recognition networks require no further processing on loading.

For efficiency during recognition, Vega employs a number of techniques, including:

- beam pruning — the deactivation of poor scoring tokens[1] in the recognition match is termed pruning [10], an extension used in Vega being ' variable beam width pruning' where the pruning criteria are adjusted depending on the recognition network activity level,
- Vega-specific memory managers to provide efficient memory allocation, garbage collection and deallocation for the large number of temporary objects such, as tokens, that are formed during each recognition,
- node input lists implemented using ordered heap — to give reasonable CPU efficiency for the simple usual situation ($N = 1$) and for the multiple candidate situation ($N > 1$),
- small tokens — tokens usually take up 16 bytes, but in Vega most tokens in the system can actually be represented using half that storage.

6. StapVoc

StapVoc constructs a recognition network from a textual list of vocabulary items (words or phrases to be recognised). The range and size of tasks with which StapVoc must deal is potentially very large.

It is envisaged that StapVoc will be used in the following scenarios:

- on-line (within a telephone dialogue) — for the generation of small, dynamically changing vocabularies, such as the names of race horses in an automated betting system,
- off-line — for the generation or update of large vocabularies, such as a list of surnames used by directory enquiries.

[1] Relative to the best token.

For on-line systems in particular, StapVoc is memory and CPU efficient. Small vocabularies can be created in fractions of a second using small amounts of memory and CPU. Large vocabularies may contain 100 000 surnames and require more memory (tens of megabytes), but can still be created within minutes.

Regardless of the specific vocabulary size or scenario, the main tasks for StapVoc remain the same, namely:

- phonetical transcription of each word — this involves creating a network of word or sub-word models representing the different possible ways of speaking that word,
- construction the recognition network for the whole vocabulary from the individual word networks,
- adjustment of the recogniser parameters, e.g. rejection levels and time-outs according to the vocabulary.

The complete process is illustrated in Fig 9.

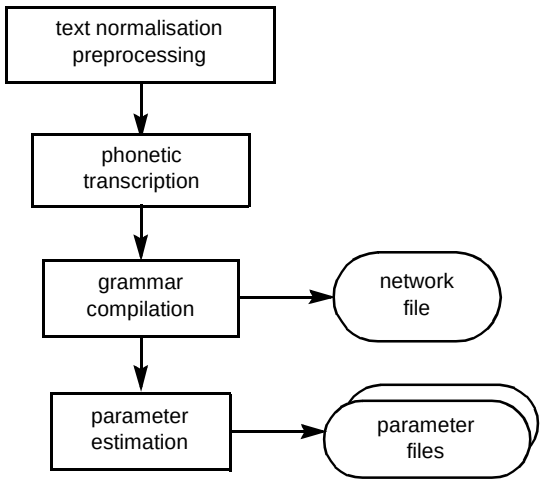


Fig 9 StapVoc tasks.

To convert vocabulary items into sequences of models each phrase is treated as a network of words. Each word is processed by first querying a ModelServer to determine whether a whole-word model can be used to represent that word. If no such model is available a phonetic transcription for the word is required. This functionality is obtained by reusing the pronunciation component of the Laureate speech synthesis system [11].

6.1 Laureate pronunciation component (LPC)

The LPC software uses lexicons and text-to-speech rules. The lexicons store words and their phonetic transcriptions and facilitate rapid look-up of transcriptions, which becomes increasingly important as the size of the lexicon grows. If the transcription for a given word is not present in the lexicons, it is transcribed using rules that attempt to mimic the process used by a person when asked to pronounce an unfamiliar written word.

6.2 Advanced facilities

For most applications, simple translation of a text string into its phonetic equivalent is just part of the vocabulary generation process. The StapVoc input file format allows an application developer access to several other useful facilities.

Aliasing

This provides a form of shorthand within a vocabulary text file. For example, in a company name recogniser, there may be a number of companies with the suffix ‘p.l.c.’. Clearly, a user may speak this in a variety of ways, including ‘limited’, ‘plc’ or ‘public limited company’. An alias can be added to the input file, so that every entry which contains ‘plc’ will be associated with all the variants.

Tagging

This enables a group of vocabulary items to return the same string when they are recognised. For example, it might be desirable to create a vocabulary in which the words ‘help’ and ‘operator’ both return the string ‘help’. This can simplify the application code without imposing any unwanted restrictions on the recognition vocabularies. Tagging effectively overrides the default labelling of paths within the recognition network.

Spelt or spoken?

StapVoc will transcribe words either phonetically or by their spelling according to its built in rules. These rules can be overridden, for example, to specify that both phonetic and spelt transcriptions should be built into the recognition network allowing for recognition of either form.

6.3 Object diagram

Figure 10 outlines the sequence of messages that pass between objects in StapVoc during the construction of a recognition network.

The Vocabulary Generation process is controlled by the VocabGenController. Essentially the following process is repeated until all vocabulary items have been processed. The VocabGenController requests a Phrase from the VocabParser which has the responsibility of parsing the user-supplied text file of items to be recognised. Phrases are added to the PhraseVocabulary which calls upon the services of the PhraseExpander to deal with, among other things, the replacement of symbols with the words they represent. Each

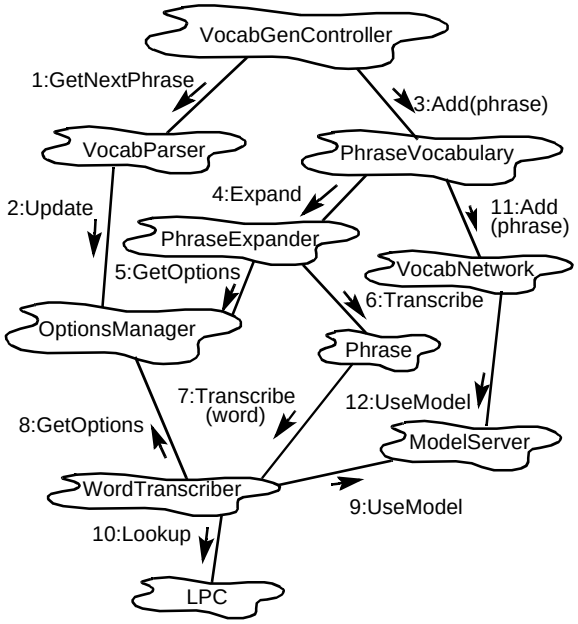


Fig 10 Vocabulary generation.

word in the expanded Phrase is then transcribed by the WordTranscriber (internally utilising the LPC software). Transcribed Phrases are added to the VocabNetwork which builds up an optimised memory-resident recognition network of all the Phrases to be recognised. The VocabNetwork queries the ModelServer to identify the most appropriate models to use in each Phrase. This architecture allows the use of whole word or subword models, and may also take into account the context of the model within the Phrase.

Once all the Phrases have been added to the VocabNetwork, the AnalysisServer scans the network in order to optimise various recognition parameters. Finally, the VocabNetwork object and recognition parameters are serialised to disk ready for use by StapRec.

7. Platforms

7.1 Stap and the ISAP

The BT/Ericsson ISAP [1] is a distributed processing system, containing a variety of VME cards, which are responsible for telephony control, signal processing and speech algorithm processing. The Stap component, StapRec and StapVoc, are examples of ISAP speech service processes (SPSs) and reside on the algorithm processor cards under the control of the ISAP process control software (SPC).

Figure 11 shows how Stap residing on the algorithm processor (AG) card interacts with a front end running on a signal processing (SP) card and with an application running on an application processor (AP) card. All of the ISAP speech services are ultimately under the control of an ISAP resource allocator (SRA) which runs on a control processor (CP) card. A typical ISAP rack will contain a mixture of AP, CP, AG and SP cards, depending on the application's profile.

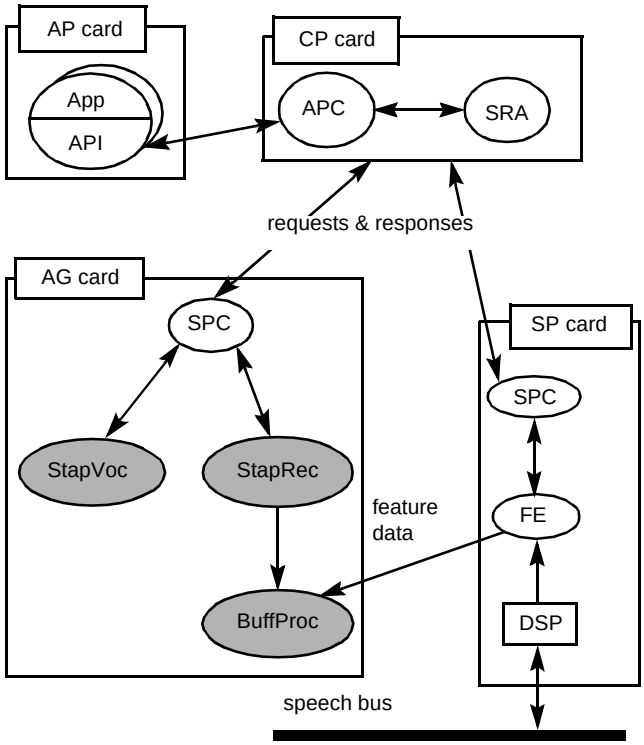


Fig 11 Interfaces between StapRec and ISAP.

7.2 Separating Stap and ISAP

ISAP processes communicate via a proprietary TCP/IP inter-process communications library (ISAP-IPC) on top of which is layered an SPC-SPS protocol. This protocol is based around a small set of messages:

- Configure — reload the configuration files,
- Do — perform a recognition or create a vocabulary,
- Interrupt — stop the current Do and return any partial results,
- Delete — terminate the Stap process.

The intention in the design of Stap was for a clean separation between the speech services provided by Stap

and the ISAP-SPS protocol, so that Stap could operate in other environments with other protocols, with little or no change. This separation was achieved by adopting the partitioning shown in Fig 12.

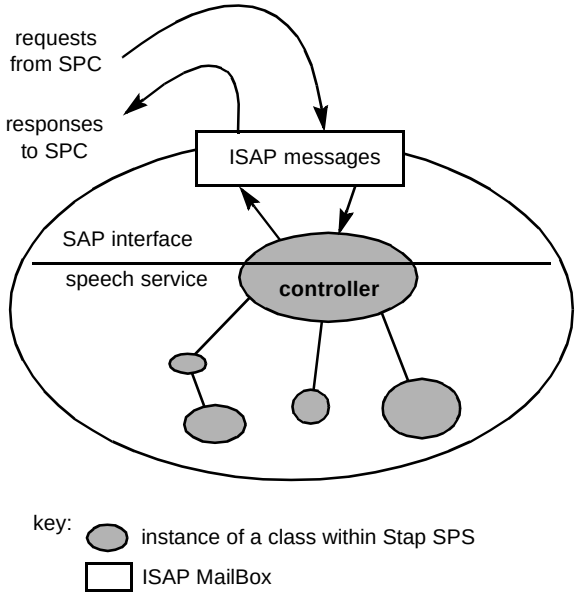


Fig 12 Separation of ISAP and speech service layers.

The ISAP interface layer constitutes an event loop which reads incoming requests from the SPC, invokes the appropriate actions on the controller, and conveys responses back to the SPC. The controller provides the public interface to the speech service (speech recognition or vocabulary generation), thereby shielding the components in the speech service from the SPC-SPS communications protocol and ISAP-IPC mechanism.

7.3 Stap for Unix workstations

Although the ISAP is the primary target platform for Stap, it was designed to be easily portable to other environments.[1]

To drive Stap in a non-ISAP environment, software known as sip was developed which takes on the roles normally provided by the ISAP. sip is written in Tcl/Tk [12] and interacts with Stap using the ISAP-IPC mechanism such that Stap needed no modifications. In fact, the same Stap executables are used in the ISAP and stand-alone workstation environments.

The main components of sip are:

- sipstap, which assumes the role of the ISAP SPC by spawning the Stap processes when required and scheduling requests for both speech recognition and vocabulary generation,

- sipfep, which provides the signal processing and feature extraction for StapRec and can accept speech data either from the audio input on the local workstation or from a file.

Workstation applications communicate with sip through Tk 'send commands' (similar to remote procedure calls). These commands for speech recognition or vocabulary creation are converted into the corresponding ISAP-IPC message and sent to Stap. Response messages from Stap are translated by sip into the return value for the Tk send.

The use of Tcl/Tk also enables easy integration of speech recognition with graphical user interfaces.

8. Testing Stap

Stap is a complex piece of software incorporating more than 60 000 lines of C++ source code. Given that this software is intended for a network platform, it has to be thoroughly tested.

Stap is functionally tested through component testing at the C++ class level and system/integration testing through a separate test harness which emulates the ISAP processes with which Stap must interface.

The system testing consists of:

- SPC-SPS protocol testing,

- recognition and vocabulary generation functionality,

- response to extreme loading,

- response to error conditions.

These tests are repeated for each platform to which Stap is delivered (currently SunOS and HP-UX).

[1] To date, Stap is available for HP or Sun workstations and has also been ported to PCs running Consensus.

In addition to confirming that Stap functions correctly it must also be established that the accuracy and resource usage (memory and CPU) lie within expected ranges. A series of tests are performed using a database of speech recordings and a variety of vocabulary sizes and types to establish recognition accuracy and resource usage statistics. Failure to meet expected performance is treated as seriously as a major functionality failure.

9. Stap in action

Currently Stap is used in a number of trial speech applications, both internally to BT as well as with some of BT's customers.

9.1 PSTN applications

The majority of applications to date have been PSTN applications [13] utilising the BT/Ericsson ISAP. They include the following.

- Advanced repertory dialling — users can store nine telephone numbers and names, the number is accessed by saying the associated name, e.g. 'Phone Home'. The user has complete control of the numbers and names and these are mixed with command words for accessing other facilities such as conference calling or voice mail. This application is an example of one in which recognition vocabularies are relatively small but change dynamically.
- Financial information services — here a caller can request information regarding their current investments. Vocabulary sizes are larger and more varied than in the repertory dialling application but tend to be less dynamic. Here the emphasis is on accurate recognition.

9.2 Workstation applications — 'What You Say Is What You Get'

In addition to the ISAP-based applications, a workstation-based application has also been under development. Stap has been integrated with the Mosaic [14] World Wide Web (WWW) browser to provide a means of navigating the World Wide Web via speech.

In this application, the hypertext links are extracted from a WWW page and passed to StapVoc to create the appropriate vocabulary configuration files for StapRec. Once StapRec has been configured, the user can drive the WWW browser simply by speaking the hypertext links. This demonstrates the speed and ease with which StapRec can be automatically reconfigured — StapRec is typically reconfigured and ready to recognise within a second or two of a WWW page being downloaded.

The architecture for the WWW application is shown in Fig 13. As can be seen, this application makes use of the sip software described earlier which allows the Stap processes to work in a non-ISAP environment, and to receive speech through the workstation audio input.

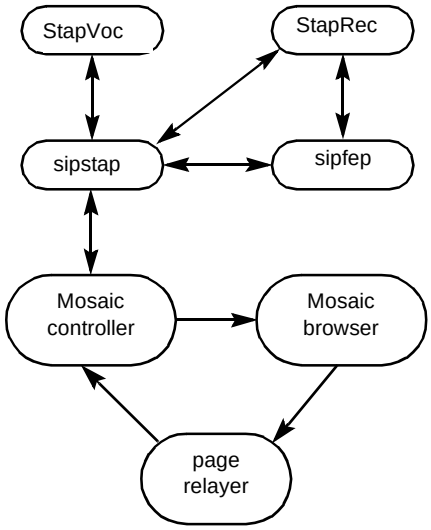


Fig 13 Controlling a Mosaic browser via speech.

This application requires a slightly modified version of the WWW browser, Mosaic, which writes the received HTML page to a Unix fifo. An ancillary process, page relayer, monitors the output from the Mosaic browser. When a complete document has been received, the entire page is sent to the Mosaic controller. On receiving the new page, the Mosaic controller parses the page to extract the subject of each hypertext link and its associated URL. Some regularisation is then performed on the link text to remove some of the unpronounceable characters and to account for the different ways in which people might speak it. The processed links are then passed to StapVoc via a file. Additional words may be added to this file to allow voice control of some of the Mosaic browser commands such as 'home page', 'forward', 'previous', 'hot list', etc. StapVoc then creates the new vocabulary from this file and finally a StapRec is configured.

The user can now speak the text associated with any of the links on the page. Speech is captured through the standard audio input of the workstation and paramaterised by the sipfep process. The paramaterised speech is then passed to StapRec in the same way as it would be in the ISAP. On completing recognition, StapRec presents the recognised text to the Mosaic controller (via sipstap) which determines the URL associated with the text and then instructs the WWW client to download the document. The process then repeats.

The major feature of this application is the close integration of the real time vocabulary generation and speech recognition tools.

10. Conclusion

This paper has given an overview of the design and capabilities of the BT real time speech recognition software suite — Stap. It has shown how the latest speech algorithms have been downstreamed into a portable, flexible product, that is being used with the BT/Ericsson ISAP as well as standalone workstations.

With Stap, BT now has the capabilility to exploit a maturing technology in real interactive speech services [15].

Appendix

A brief introduction to the Booch notation

Booch has developed a notation for capturing and communicating the analysis and design decisions that are made during the development of complex object oriented systems. These include, the taxonomic structure of the classes, the inheritance mechanisms, the individual behaviours of objects, and the dynamic behaviour of the system as a whole.

The Booch notation is expressive and quite detailed. However, a small subset is quite adequate for expressing most of the analysis and design issues. The notation defines the following views:

- class — show the existence of classes and their relationships in the logical view of a system,
- object — show the interactions that may occur among a given set of class instances,
- module — show the physical layering and partitioning of the architecture,
- process — show the allocation of processes to processors.

To illustrate this notation, a class and object diagram for an artificially simple scenario are shown in Figs A1 and A2 respectively.

Fig A1 shows that a Garden ***has*** one or more Plants and ***uses*** a GardenCentre. The ***has*** relationship denotes a whole/part relationship and appears as an association with a filled circle at the end denoting the aggregate. The class at the other end denotes the part whose instances are contained by the aggregate object. The ***uses*** relationship denotes a client/supplier relationship and appears as an association with an open circle at the end denoting the client.

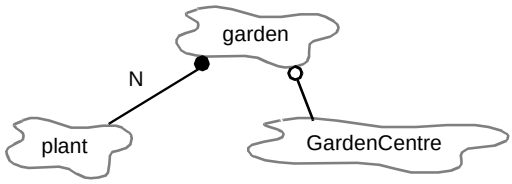


Fig A1 Class diagram.

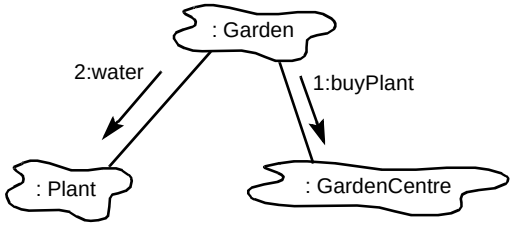


Fig A2 Object diagram.

Fig A2 illustrates an instance of Garden, invoking the water method on one of its Plant instances, and invoking the buyPlant method on its GardenCentre instance.

References

1 Rose K R and Hughes P M: 'Speech systems', BT Technol J, 13, No 2, pp 51—63 (April 1995).

2 Cox S J: 'Hidden Markov models for automatic speech recognition: theory and applications', BT Technol J, 6, No 2 pp 105—115 (April 1988).

3 Booch G: 'Object oriented analysis and design with applications', (2nd Ed) Benjamin Cummings (1991).

4 Stroustrup: 'The C++ programming language', (2nd Ed) Addison-Wesley (1991).

5 Power K J: 'The listening telephone — automating speech recognition over the PSTN', BT Technol J, 14, No 1, pp 112—126 (January 1996).

6 Pawlewski M P et al: 'Advances in telephony-based speech recognition', BT Technol J, 14, No 1, pp 127—150 (January 1996).

7 Attwater D J and Whittaker S J: 'Issues in large-vocabulary interactive speech applications', BT Technol J, 14, No 1, pp 177—186 (January 1996).

8 Ringland S P A: 'Grammar constraints through signatures', in Rubio A et al (Eds): 'Speech Recognition and Coding: New Advances and Trends', Springer (1995).

9 Young S J, Russell N J and Thornton J H S: 'Token passing: a simple conceptual model for a connected speech recognition system', CUED technical report (1989).

10 Steinbiss V, Bach-Hiep T and Ney H: 'Improvements in beam search', ICSLP'94, 4, pp 2143—2146 (1994).

11 Edgington M E et al: 'Overview of current text-to-speech techniques: Parts I and II', BT Technol J, 14, No 1, pp 68—99 (January 1996).

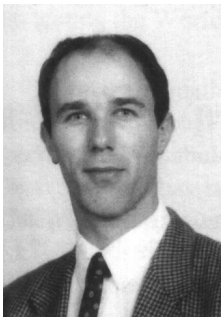
12 Ousterhout J K: 'Tcl and the Tk toolkit', Addison-Wesley (1994).

13 Talintyre J E and Ringland S P A: 'Isolated word recognition over the PSTN', IOA Autumn Conference (1989).

14 National Centre for Supercomputing Applications, NCSA Mosaic™ for the X-Window System', http://www.ncsa.uiuc.edu/SDG/Software/XMosaic/help-about.html/

15 Whittaker S J and Attwater D J: 'Interactive speech systems for telecommunications applications', BT Technol J, 14, No 2 (to be published, April 1996).

Frank Scahill graduated from Brunel University in 1989 with a BEng in Electrical and Electronic Engineering. He then joined the speech technology unit of BT Laboratories to work on various aspects of speech recognition research and development. He was a member of the team that developed the speech recognition algorithms and software in the BT Call-Minder service. He currently leads the software development of the BT Stap speech recogniser.



John Talintyre received a BA degree in electrical and information sciences tripos from Cambridge University in 1987. He joined BT Laboratories later that year and began work on speech recognition research and in particular the recognition of the digits and yes/no from any UK speaker across the PSTN. In 1991 he helped to start up a group for developing speech recognition algorithms which he still heads. Small and large vocabulary speaker-independent recognisers have been produced as well as a speaker dependent speech recogniser and the ability to generate speech recognition vocabularies from plain text.

Stephen Johnson joined BT Laboratories in 1989 after graduating with a BSc (Hons) in Electronic Communications Engineering from the University of Hull. He spent his first thee years working in speech recognition research concentrating mainly on techniques for improving feature extraction. He then moved into the development of speech recognition algorithms for deployment in services such as the BT CallMinder. He has played a significant role in the development of the BT Stap recogniser leading the design and development of the vocabulary generation component, StapVoc, as well as the software development for Stap version 2.

Tony Bass has a Cambridge University MA degree in mathematics. He joined the Post Office in 1970 and worked for many years on software for optical character recognition and then later for natural language translation.

He is now in the recognition development team working at BT Laboratories on speech recognition software.

John Lear received an MEng (Honours) degree in Electronic Systems Engineering from Aston University in 1994.

On leaving University he joined the recognition development team at BT Laboratories initially working on HMM training software and then the Stap recogniser.

David Franklin graduated with a first class honours degree in general engineering from Surrey University. On graduating, he worked briefly for BT before taking a PhD in microprocessor engineering at UMIST.

Since then he has worked for a wide range of software companies in areas which include image analysis, expert systems, hypermedia and, more recently, speech recognition and distributed computing.

Philip Lee graduated from the University College of Swansea in 1988. He then joined AT&T ISTEL where he trained as a software engineer and went on to provide technical direction to several major projects. He currently works as a freelance software consultant specialising in object oriented design and development.